key thing to note here is the component has already been rendered, so the display state is still true from the previous render; the useState hook doesn't reinitialize. The useEffect does set animateOut, so the out-animation CSS is added, causing the element to slide out to the right. The onAnimationEnd event then fires, setting the display to false and causing the component not to display. On future renders, display is already false and shouldBeVisible will also be false, so the element won't display at all and it won't matter that the animation CSS is there.

This gives us the behavior we want: new favorites animate in, removed ones animate out.

ActionCable and React

This page needs two ActionCable subscriptions: the one that registers a ticket hold on a different page and updates the tickets remaining, and the one that registers the user changing their favorites in a different browser.

The ticket update subscription is more straightforward. It's already being broadcast by the concert display page and already sending JSON data, all we need to do is catch it. Here's the function we called way back when the ScheduleApp component was created:

appendix_react/01/app/javascript/contexts/schedule_context.ts

```
export const initScheduleChannel = (): ScheduleThunk => {
 if (scheduleChannel !== undefined) {
  return
 }
 scheduleChannel = createConsumer().subscriptions.create(
  "ScheduleChannel",
  {
   async received(data) {
    const parsedData = (await data.json()) as ScheduleChannelData
    parsedData.concerts.forEach((concertData) => {
     scheduleStore.dispatch({
      type: "updateConcertRemaining",
      concertId: concertData.concertId,
      ticketsRemaining: concertData.ticketsRemaining,
```